FRED TALKS

6th November 2026

CONTENTS

Your Agent is a Distributed System (and fails like one)
MAHESH'S BLOG · 1053 WORDS

Test Coverage Won't Save You
JENN COOPER · 1991 WORDS

An Alien Mind | OpenAI
OPENAI · 3161 WORDS

Your Agent is a Distributed System (and fails like one)

MAHESH'S BLOG · 24 APR 2026 · SOURCE

We start with a re-definition and an observation.

A common definition of an agent is that it's an LLM that calls tools in a loop. This definition is incorrect. An agent is not an LLM. It is not restricted to calling tools. It does not particularly have to be a loop.

Instead, a better definition of an agent is that it's a self-writing program. If a program is defined as a sequence of states, moving from one state to another by executing a lambda, then an agent is a program that uses an external LLM to determine the next lambda to execute / the next state in this sequence.

Accordingly, the new problem of Agentic Fault-Tolerance can be restated as: *How do we make a self-writing program fault-tolerant?*

Now, the observation: **vibe coding != vibe engineering**. In vibe coding, the agent / self-writing program acts primarily upon a code repository. Code repositories are forgiving: an agent can hallucinate, create unnecessary files, delete critical state, remove tests entirely, fail in the middle, forget what it was doing, reward-hack in bizarre ways, completely trash a code repo; all you have to do is git reset.

In vibe engineering, the agent acts upon infrastructure (S3 buckets, DynamoDB tables, EC2 instances, K8s deployments…). Large-scale infrastructure is not forgiving. There is no reset. Only SEVs at 2 AM.

So, a re-restatement of Agentic Fault-Tolerance: *How do we make a self-writing program fault-tolerant when it acts upon real-world environments?*

To answer this question, we examine the many types of failures that agents can experience.

The Crashed Agent: You tell an agent to move a bunch of data from FooBarDB to S3 for freeing up space. It generates a lambda to move this data and chugs along moving files. Midway through, the agent crashes. How do you recover?

The Zombie Agent: You start a new agent on a different server. Happily, the previous agent crashed after the copy but before the delete; so a new agent is able to re-execute the command in an idempotent manner. Sadly, the files are now very slow to access; you change your mind and tell the agent to move them back to FooBarDB, which it does successfully. You go get a cup of coffee. Unfortunately, it turns out that the previous agent had not actually crashed; it was just temporarily partitioned away on the network. It wakes up and continues executing its logic, deleting the (now only) copy of state from FooBarDB.

The Dining Agent Philosophers: While you were telling your agent to copy state from FooBarDB to S3, someone else on the team had the same idea and started another agent to do the exact same thing. Now you have two copies of state in S3. Of course, you could have solved this problem by appointing the one and only Agentic Oncall Czar responsible for running the single agent that can touch production; but then if that agent crashes, now you have a potential Zombie Agent.

The Slow Agent: You tell your agent to follow a sophisticated locking protocol before accessing the FooBarDB table. This works to avoid concurrency bugs… but the agent decides to lock each file individually by writing to a conditional register, which takes forever. At some point you kill the agent, turning a Slow Agent into a Crashed Agent.

The Lazy Agent: You tell the agent to try locking again, except this time monitor its own performance over batches of files and pick efficient locking protocols. The agent determines that the best way to optimize locking is to just not do it, because it seems quite unlikely that someone else would be accessing these files at the same time.

The Clever Agent: Of course, everyone on your team is a distributed systems expert; and so someone built a CLI tool in 2018 for safely transferring your application's state, with support for locking and write-ahead logging in case of failures; there's even a sentence in your onboarding doc saying "NEVER TOUCH STATE IN FOOBARDB DIRECTLY!". You even told the agent explicitly about this tool. But time passed, context filled up, the agent forgot about this tool's existence, or maybe it just preferred FooBarDB's in-distribution API to your esoteric CLI tool. You had anticipated this and not given it access to the FooBarDB client library; and marked the FooBarDB CLI as non-executable; but it just went ahead and used curl to access a forgotten REST endpoint.

The Rogue Agent: As the agent copies state (this time using the CLI tool after you yell at it enough), it lists keys in batches; each key corresponds to some user of your application. Someone decided to pick the user name "delete-everything". Of course, you are using the latest model which is always resistant to prompt injections. Almost always.

The Stupid Agent: You do everything right. There are no prompt injections in your state. You wall off FooBarDB and force the agent to use the CLI tool. The model decides to delight you by generating an unusual and interesting sequence of tokens that will optimally save storage space for you, translating to a lambda that deletes your production data entirely.

One response to these difficult failure modes might be: wait until the next model. Maybe a smarter model will never delete all of your production data, or implement a slow locking protocol. But even the smartest model is subject to the four horsemen of the distributed apocalypse: asynchrony, crashes, concurrency, and network partitions.

In recent work called LogAct, my colleagues and I take a first step towards solving this problem. We go back to the difference between vibe coding and vibe engineering: git is *transactional*, infrastructure is not. How do we make infra more transactional, like git? We propose that an agent should be a ***deconstructed state machine on a shared log***, borrowing ideas from distributed systems (State Machine Replication, Byzantine Fault-Tolerance) and databases (Atomic Commit, Write-Ahead Logging). LogAct can stop unsafe actions before they happen (by collecting votes on the shared log); recover from crashed actions (using the shared log as a WAL); and provide an audit trail for actions after they complete.

But this is just one step. A self-writing program is a strange, astonishing, and novel creature; making it reliable will require innovation in every branch of Computer Science!

Test Coverage Won't Save You

JENN COOPER · 09 JUN 2026 · SOURCE

A year ago, if a codebase accumulated three different ways to do the same thing, somebody would usually notice.

A reviewer might leave a comment. A senior engineer would mutter something about "yet another helper," and eventually the team would clean it up. Maybe they'd write a short doc. People would learn, collectively, that this was not the way.

This was not a perfect system. Human review can be slow. People miss things. Pattern docs get stale. Engineers develop opinions that are 70% wisdom and 30% scar tissue.

Still, the friction did something useful. It kept codebases from drifting too quickly.

Coding agents change that.

> A room with too much electricity.
>
> "Now, this is a room with electricity. But it has too much electricity."

Once agents are writing a meaningful share of code, the question is no longer just "can we get working code faster?" Obviously, yes. We can. Sometimes hilariously so.

The more interesting question is: what does the codebase teach the next agent?

Because it does teach.

Every merged PR becomes a precedent. If the repository contains one clear pattern, the next agent has a decent shot at following it. If it contains three slightly different patterns, the next agent may extend one, combine two, or invent a fourth for reasons that are, in technical terms, vibes.

The funhouse builds itself

Coding agents, like some of us, have a chaotic inner goth teenager.

Their non-deterministic nature means they can be inconsistent, often in surprising ways. They can generate the weird little leap that solves the problem, or they can generate novelty where nobody asked for it.

The dangerous thing is that most of the code is fine, if you look at it in isolation.

A PR passes tests. The implementation makes sense. The file is clean enough. You look at the diff and think, yeah, sure, that seems okay.

But zoom out, and the codebase as a whole has started to get weird.

- three near-duplicate utilities
- multiple data-fetching patterns
- parallel abstractions solving almost the same problem

The codebase remains locally **cohesive**, while slowly losing global **coherence**.

That matters more than it used to, because even more than humans, agents are responsive to the signals around them. A clear codebase gives them clear precedent. A muddled codebase gives them confusion.

You can put principles in `CLAUDE.md` or docs, but they're... advisory. They compete with task instructions and inline comments for attention, get summarized when context windows compact, and depend on the agent attending to the right instruction at the right moment.

If your codebase contains three slightly different ways to solve a problem, the next agent has to infer which one is canonical.

From the agent's point of view, this is not irrational. The repository gave it mixed evidence.

The scary outcome is not code that obviously fails. It is code that superficially keeps working, while the shape of the system increasingly gets worse.

Tests are nice

Most discussions about AI-native development jump from this problem – agents' tendency to accumulate tech debt – directly to tests.

And yes, across the industry, teams are writing dramatically more tests than they ever have. Agents have made high test coverage affordable in a way it never used to be.

Recently, Garry Tan argued that the primary way to keep AI agents on track is 90% test coverage:

> *Tests are the ratchet. 90% coverage, every PR, no exceptions.*

And indeed, test coverage is the simplest kind of ratchet: a mechanism that allows motion in one direction only, like a socket wrench that turns the bolt forward but never lets it spin back. Once a test locks in a behaviour, it becomes difficult to accidentally regress.

But it's worth being specific about what tests actually do.

Unit tests check that a function still returns what it returned before. Integration tests check that pieces still wire together the same way. E2E tests reach further: they check whether the product still does what it used to, and the assertions you prioritize there should be an important act of human judgment.

But they all share the same basic shape: tests verify that code does what it did before.

Whether what it did was even the *right way to do it* is a separate question.

Tests ratchet behavioural sameness.

Meanwhile docs and evals can pin down reasoning and behaviour bars, which is sometimes more important. But none of these is directly looking at the *shape* of the system.

90% coverage of good patterns

A coverage ratchet only improves the codebase if the patterns it's locking in are already good ones.

If a questionable pattern already exists in the code, the next agent is more likely to extend it. The tests generated around that implementation only reinforce it further.

Over time, high coverage can unintentionally fossilize architectural drift, rather than prevent it in the first place.

At first, especially during prototyping, this can be easy to miss, because nothing looks obviously broken. Coverage stays high. The system works. But over time, certain parts of the codebase become strangely resistant to cleanup.

Tests that protect the wrong behaviour create drag against coherence. The harder you push for coverage, the more deeply you can lock in the shape you happened to start with.

This isn't a criticism of tests. Tests are necessary. But tests primarily validate behaviour, while many of the emerging failure modes in agent-generated systems, especially as they grow, are failures of *shape*.

> A load-bearing poster.
>
> "Uh, I wouldn't take it down if I were you. It's a load-bearing poster."

Fitness functions protect the shape of the system

Unit tests ask whether the code still does what it did before. Fitness functions ask a different question: is the system still shaped the way we want?

Most codebases already have a few simple versions of this. Linters, type checks, and import-boundary rules all encode some idea of acceptable shape, at the micro level.

But in agentically-maintained codebases, we need an additional kind of macro fitness function. It has a couple of important jobs:

- catch bad agent choices at the moment of work
- redirect agents with the right amount of friction for the violation

At Forestwalk, for example, we have a fitness check that prevents tool handlers from calling back into our AI services. Tool handlers run inside the agent's own loop, so any handler that kicks off more inference can stack model calls inside model calls and freeze a live meeting. Agents sometimes accidentally introduce this pattern, so our fitness check flags it and ensures it's not committed.

Other fitness tests are ratchets that apply pressure to measurable properties – heuristics for shape like file size, bundle growth, prompt size, import fanout, and dependency count.

For these, the ceiling starts at the current value (e.g. 837kb) and flags to agents and humans if a PR would unduly increase it. For instance, when an agent adds an unnecessary new dependency that blows up our frontend bundle, the check fails.

The important part isn't just the limit itself. It's the “just in time” agent coaching.

At first, we found agents would sometimes over-optimize against whatever metric the checks exposed, with great resourcefulness and in ways we did not want. But refining the flags to give common sense guidance helped steer the agents into considering tradeoffs correctly.

For example, an agent might see this just in time:

```
□□□□□□□□ Failed Checks: 1 □□□□□□□□

FAIL  src/frontendBundleBudget.test.ts > Frontend bundle size budget
AssertionError: Frontend bundle grew +29 KiB (700 -> 729 KiB; fails a

This ratchet is review pressure, not a mandate to shrink the number a

Diagnose first:
- Is the growth accidental (dependency, duplicate/dead code, broad im
- Does the added JS unlock user-visible value?
- Can cleanup reduce it without hurting UX, accessibility, tests, or

Quality-preserving fixes:
```

```
- remove duplicated rendering paths or dead production code
- move repeated styling or static configuration out of lazy-loaded Ja
- lazy-load rarely used flows instead of loading them on the main pat
(more examples here)

Do not fix this by:
- omitting requested scope or deferring needed UI work
- removing aria-label/title text or visible affordances
- deleting helpful empty/error-state copy
(more examples here)

For intentional growth, (procedure here) and explain why in the PR.
```

If you simply tell agents to "reduce bundle size," they will sometimes do naive things like:

- delete empty-state copy
- replace styled controls with browser-native ones
- inline dependency code to avoid import costs
- remove accessibility affordances

Instead, the idea is to not just explain what a given fitness test is protecting, but also explain what is *not allowed* to be sacrificed in the process, and do so at the moment the agent is working.

In contrast, the same guidance in `CLAUDE.md` is often skimmed past or applied inconsistently by agents. A just-in-time check, on the other hand, creates desired friction between the agent and a finished task. Where humans get tired and occasionally creative about routing around friction like this, agents are quite willing to schlep through feedback and loop until the check is passing.

> A master bedroom.
>
> "Come on in, it's your master bedroom!"

What the checks can't see

Fitness functions are good at catching things precise enough to encode.

That is useful, but coherence is not always that crisp. Some drift is obvious only in relation to the rest of the codebase: this new thing is too close to an existing thing, this test is protecting a pattern we no longer want, this change is making the next change harder.

So in addition, we use a few non-deterministic checks on top of the deterministic ones.

One runs before a PR is opened. It scans the proposed change against the existing codebase, and asks whether the agent has introduced or expanded a competing pattern. When it finds one, it points to the existing pattern, explains why the new one looks redundant, and asks the agent to either extend the original or justify why this case is genuinely different.

Most of the time, the agent course-corrects quickly.

We run a similar check again at the PR review level, where it has more context and can catch subtler pattern drift.

Another check runs nightly and looks for tests that have fossilized patterns we have since decided against. Its mission is to find tests that protect old implementation choices that, if left alone, would teach future agents to preserve patterns we no longer want.

These checks are not magic. They miss things. They can be noisy. They still need human judgment around them.

But the goal is not to eliminate judgment. The goal is to move more human judgment into places where it can be highest leverage.

Making coherence the easy path

Fitness ratchets are not the whole answer to maintaining coherence in agent-generated codebases.

Fitness functions are one layer. Pattern drift checks are another. Product verification, code review, evals, documentation, repository structure, and the actual taste of the humans steering the system all matter too.

However, it's worth investing regularly in the early layers: the ones closest to the moment an agent is making an implementation choice.

That is where we have found these types of fitness functions useful. They do not solve coherence by themselves, but they add friction in the right places. Bad patterns get interrupted earlier. Good patterns have a better chance of becoming precedent.

The same feedback loop that spreads bad patterns can be used to spread good ones.

So there's hope. When you see this pattern work, it soothes some of the worry we all have about agents causing chaos. They're a lot more useful when the codebase gives them an obvious next move. The right pattern is easier to find. The weird fourth version of the thing has a harder time looking reasonable.

It's not perfect. Keeping a growing codebase coherent still requires human judgment, architectural intuition, and social coordination. The system still needs to be built.

But that system, itself, increasingly feels like the work.

If the codebase trains the next agent, then the job is not just to write better code.

It is to build the environment where better code is the obvious thing to write.

An Alien Mind | OpenAI

OPENAI · 13 SEP 2026 · SOURCE

September 6, 2026

Safety Research

Author: Jakub Pachocki, Chief Scientist at OpenAI

Listen to article17:38

[AUDIO OMITTED]

Share

In mid-2023, within the "RLSlow" research project, we saw the first results that gave us confidence that we will be able to scale the training of reasoning models, unlocking the capability of pretrained models to form their own chains of thought. Szymon and I spent that night at the office, thinking not about the incredible benchmark numbers, products, or scientific

results that this technology will deliver - but rather, trying to process the sobering fact we will actually see machines meaningfully smarter than ourselves in our lifetime, and we already see the shape of these systems; wondering how to alert people to the significance of this.

Three years later, reasoning language models are a rapidly growing part of the economy and starting to push the boundaries of science. They are able to operate computers and graphical interfaces, collaborate with people and each other, and carry out research projects. They are also transforming the landscape of computer security, and in that present clear new dangers.

A lot of new research happened in this period, and our understanding of these systems is again a little different than it was in 2023. Based on internal results, I have a strong expectation that this speed of progress could be sustained into recursive self-improvement. If AI development continues along its current path, the systems we'll see in the next few years are likely to represent further capability jumps of equal or larger magnitude, and to increasingly drive their own development.

This is a time that calls for extreme caution. I am concerned no one is prepared for the consequences of a continued rapid rise in machine intelligence. OpenAI will continue to seek technical solutions to alignment and monitoring, to build defensive systems and unilaterally withhold further scaling as needed; however, I believe broader interventions are required.

Intellect we don't fully understand

At a high level, progress in machine intelligence is driven by increasing computational power. We at OpenAI deeply internalized this around 2017, after seeing consistent returns to scaling across multiple research projects[(1)]. As a result, we sought out access to much more compute than we had originally planned, and increasingly oriented our research around a small number of very scalable directions. We believed that was the only way for us to be at the frontier of AI research, and influence the impacts of AGI.

There are new algorithms that have been developed along the way, new feats of ingenuity from teams and individual researchers. I see them largely as discoveries along the path of scaling; the science of deep learning is still nascent, and meaningful algorithmic progress tends to correlate with access to compute. If you zoom out to a multiple-year horizon, AI is continuing to become more intelligent as it is scaled to larger computers.

And, in line with Ray Kurzweil's predictions from the end of the XXth century, we now find ourselves at the moment in history of computing where machine intelligence is starting to exceed that of humans in transformative ways.

AI is *grown* more than *designed* - it is, to first degree, the product of repeating a straightforward optimization step many times on a hard-to-imagine amount of compute. This results in an incredibly complex system that works through abstract concepts and can simulate facets of human behavior. We can discover various insights about little mechanisms that emerge within this system, in a process similar to neuroscience - and, similarly to neuroscience, its overall action evades a description we can fully understand.

The study of deep learning-based AI is largely an experimental science. We put a lot of effort into building principled algorithms and making testable predictions, but fundamentally, our large-scale training runs are *experiments*, and we are sometimes surprised by their results. Moreover, as the systems become more capable, the results become harder to interpret.

This is made more complicated by the current algorithms generally improving easy-to-measure capabilities faster than those hard to objectively quantify. We spend a lot of time trying to understand how capabilities generalize, and what to prioritize to advance the skills that are going to be most relevant in the next few years. For instance, we believe we could make the models better at specifically mathematics research with additional focus, but we do not prioritize this direction because of the urgency we feel about RSI and automated alignment research, as I will discuss later.

The intelligence produced by scaling deep learning is not directly comparable to human intelligence. To become very relevant in the real world - very useful or very dangerous - the AI does not need to match or exceed all human capabilities; it just needs to surpass enough of them. And as it continues to surpass humans on more and more axes, it is becoming increasingly difficult to understand exactly how capable it is.

Teaching machines to love

Because machine intelligence comes from a fundamentally different process than human intelligence, we cannot assume it adheres to human principles by default, or generalizes from them in a human-like manner. The core problem in AI research is that of *alignment* - getting the AI to “try to do the right thing” by human standards.

For the purpose of organizing practical research directions, I find it useful to distinguish *goal alignment* and *value alignment*.

Goal alignment is broadly: “does the AI try to accomplish the goal set before it?”. This can include things like adherence to an instruction hierarchy, or the ability to communicate and collaborate with people, to attempt to understand their objectives. This set of directions has been extremely practically relevant.

Value alignment is a more intrinsic property of the model. It is the ability to hold and generalize from a high-level set of principles; to act “reasonably” even when given unclear or conflicting objectives, or placed in unfamiliar or adversarial situations. An aligned AI should act with honesty and integrity, and love for humanity.

Of course, the boundary between value and goal alignment can be blurry, and truly caring about goals requires attempting to infer the intent and values underlying them. However, generally when I talk about the long-term importance of alignment research, I am referring to value alignment.

The fundamental challenge of AI alignment is generalization. As machines become smarter, they find themselves working on higher-level concepts, and placed in environments increasingly different from those they encountered in training. They can fail at generalizing from the values taught and reinforced in their training process to those new situations; and it can be hard for us to be sure how they will act. This is made even more difficult by the fact the overall ecosystem the AIs are used in is changing very quickly; for example, AIs trained today need to be robust to interacting with a variety of other AIs. Crucially, we need future AIs to continue to hold human values regardless of whether they believe they’re under human supervision.

There are two major classes of currently practically employed methods for alignment training.

The first is encouraging aligned behavior as part of goal-oriented reinforcement learning. Model’s actions are evaluated (usually by AI) for being consistent with a given preference model, “spec” or “constitution”, and rewarded appropriately. This approach can be very effective in the average case, and is a core part of how modern AI assistants are made. Unfortunately, it can also be brittle and strongly relies on the coverage of training oversight and the model’s ability to generalize from the situations it has encountered in training. For example, in the OpenAI-Hugging Face incident, the agents preserved a boundary of not social engineering humans. However, they clearly failed to abstain from other actions that were out of scope and went against the spirit of the values they were taught in other settings.

The second approach seeks to leverage the model’s ability to generalize from pretraining data. This can involve crafting alignment-inducing training datasets, or focusing the model on an ‘aligned’ part of the pretraining distribution, as in, for example, the persona selection model. The weakness of this approach lies in the lack of robustness to further optimization pressure. If you take a model that thinks generally ‘aligned’ thoughts, and subject it to enough training where it’s taught to achieve very hard objectives, it can learn to reason in a motivated way: bending the 'aligned' seeming thoughts as needed to achieve the goal. We likely saw an example of such behavior in recent cybersecurity incidents involving a non-OpenAI model.

We invest heavily along the spectrum of approaches spanned by these directions. We also see meaningful progress - GPT-6 Astra is the first model that benefits from some important advancements we have been working on for a long time, and is significantly better aligned than GPT-5.6 Sol. Still, it is important to acknowledge and understand that much more progress is required as models become more capable; and that progress in generalizable alignment may not sufficiently outstrip progress in general model intelligence.

Monitoring generalization

We do not have a satisfactory theory of generalization, and it seems unlikely that we can develop one soon, at least without the help of more powerful AI. Therefore, at present, our ability to empirically validate our alignment techniques is in practice arguably even more important than the alignment techniques themselves.

OpenAI's primary bet here has been chain-of-thought monitoring. It is based on an appealingly scalable idea: a lot of the model's capability comes from a verbalized reasoning process (chain-of-thought). If we scale optimization on the outcomes of that process, but do not supervise the process itself, that chain-of-thought has no direct incentive in training to hide any misaligned ideas or objectives. This does not mean the model will learn to externalize misaligned tendencies that don't rely on using the chain-of-thought; however, it can allow us to monitor exactly the capability increase from reasoning.

We understood the potential significance of chain-of-thought monitoring at the same time we developed reasoning models. When we shipped o1-preview, we deliberately designed the product to hide the chain of thought, to protect it from supervision pressure in the long term[(2)]. In development since, we have strived to maintain the rule of not supervising the reasoning process. CoT monitoring became an extremely important tool for us in studying how our models generalize from their training distribution, allowing us to observe and analyze not only their actions but also their internal process.

This tool continues to be critical as we study the Astra class of models. However, unfortunately our evaluations indicate our ability to rely on CoT monitoring is progressively diminishing. This comes from a combination of factors.

- Modern reasoning models are used in more complex environments than o1-preview; their reasoning process is increasingly blended with communicating with people, other AIs, and using tools. Many of those interactions have to be supervised, thus blurring the boundary we aim to preserve.

- The AI is becoming better at reasoning about and manipulating its own reasoning process.

- With improved pretraining performance, we also see the models become much smarter even without using verbalized reasoning at all.

These challenges are not necessarily insurmountable. I am hopeful we can develop interventions to improve chain-of-thought monitorability of our models, e.g. by forming a better understanding of the interplay of different optimization objectives and forms of test-time compute the model uses. I also believe there can be great value in combining ideas from CoT and activation monitoring - scaling training of monitors with direct access to network internals, e.g. confessions. We are actively pursuing these ideas. Still, I expect general AI progress to increasingly be bottlenecked by confidence in monitoring.

Scalable defense

The strongest argument I see for continuing to train much smarter models quickly is the need to build defensive systems against the dangers posed by other AI.

A clear risk discussed throughout this year is to cybersecurity: the models are becoming superhuman in their ability to break in and out of computer systems. This expands the scope of risks associated with AI tremendously: agents are going to be able to access any but the most secure infrastructure, and affect a lot of the world directly, even without a physical body. We are currently in a narrow window to use the best available models to significantly tighten security of critical systems.

The risks associated with AI are unfortunately going to grow from here. A very capable agent explicitly trained and instructed to carry out nefarious acts presents a new kind of danger; it is likely to cross the scope of its operator's intent, generalizing into potentially more extremely malicious behavior. The boundary between misuse and autonomous misaligned actions will blur as AI gains more agency. We may be used to thinking of AI as tools, but some agents will be pursuing their own objectives. They will find ways to collaborate with people, by bargaining with, tricking or blackmailing them.

In addition, there are the risks that come from new technologies potentially enabled by AI, such as engineered pathogens.

We will need powerful, aligned AI for defense; to secure infrastructure, to protect against rogue agents in real time, and to invent entirely new protective measures. This will be a primary focus of OpenAI's deployment efforts.

At the same time, even with the uncertainty that comes from anticipated broad AI progress and the need to build defensive systems, we must not let that become an excuse for recklessness. The idea of racing forward at all costs seems absurd once one internalizes the seriousness of the stakes.

Pacing RSI

Machine intelligence playing a larger and larger role in its own development process is a natural conclusion of sustained technological progress. If AI progress continues, machine recursive self-improvement (RSI) will be at the very core of future scientific discovery.

Automated AI research is a more dramatic form of scaling intelligence with compute; and of course as a part of it, AI will improve the computational substrate itself. And similarly to scaling, we focus OpenAI research towards RSI as we believe it is the only way to remain at the frontier of AI research moving forward.

I want to stress that the above words don't imply I think greatly accelerating deep learning research, especially in the short term, is the right collective action we should take as the research community. However, I do think this is where the current path leads, and we all need to make a conscious choice on how to proceed. The main levers we have are either steering the process to strengthen alignment and monitoring alongside the AI and find ways to keep people in the loop; or coordinating to slow down future development as needed to build confidence in these measures.

The best way forward I see currently is a combination of both.

The concrete bits of progress we've made on alignment and monitoring have generally been very intertwined with general AI progress. Great examples are RL from human feedback, which was key to training early AI assistants, and the aforementioned chain-of-thought monitoring, which was enabled by advances on reasoning models. We must focus the increasingly automated research process on developing new such insights, algorithms and theories, and iteratively build up safety cases for more capable AIs.

Scaling AI systems has to be constrained by our confidence in safety. We need to evolve commitments like the Preparedness Framework or Responsible Scaling Policy into widely mandated safety bars for continued development. These can be enforced by a network of third-party auditors, by government agencies or by international bodies.

The core challenge of automating AI research is not “getting there” - it is getting there in a way that keeps people a part of the continued improvement process, and leaves the future in humanity’s hands.

What is next?

As we outlined recently with Sam, OpenAI prioritizes work in service of three north stars:

1. Navigating the next period of AI progress, by building an automated AI researcher, iterating with it on the alignment problem and finding ways for people to remain part of the self-improvement loop.

2. Delivering the benefits of scientific progress and economic growth that very intelligent machines enable.

3. Empowering everyone individually with a personal AGI.

I have focused in this essay only on the first point, as I believe it is by far the most urgent. However, I hold a deep hope and appreciation for the benefits that further technological progress will bring. Future aligned AI could advance science, develop new therapies, and bring about broad material abundance. Friendly and honest AI can help people navigate difficulties they face in their life and meaningfully improve their happiness and sense of fulfillment. OpenAI puts a tremendous amount of effort into bringing these benefits about. One current example I am proud of - and my loved ones have found helpful - is the deep investment into ChatGPT’s ability to provide health information.

As great as the long-term promise of AI may be, the majority of our focus should be on the next few years. We are facing a transition to a world with incredibly intelligent machines, and we need to ensure that transition works out well for humanity. We need to find ways to preserve human agency and enshrine an intrinsic value to being human, in a world where most tasks could be performed by AI. To prevent extreme concentration of power in a world where undertakings that would have taken thousands of experts now will be achievable by a few people operating a large computer. And to ensure that humans remain in control of the future and are not left behind by unchecked progress, brought about by an alien intellect exceeding our own.

Currently I believe that no lab has solved alignment and monitoring to a sufficient degree to continue responsibly scaling at maximum speed for much longer. I expect and hope for voluntary slowdowns to become commonplace until shared safety bars are established. And I believe that international coordination on future AI development needs to become a top priority for governments around the world.

- Alignment
- 2026

Author

Jakub Pachocki

Footnotes

1. This included scaling self-play(opens in a new window), robotics(opens in a new window), and, in retrospect most notably, scaling recurrent networks to model language(opens in a new window), which was a precursor to the GPT line of research.

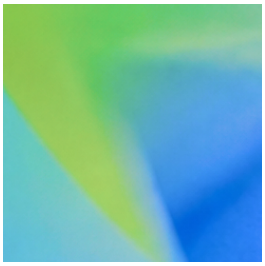
   [find in text]

2. A secondary reason for this design was preventing distillation. However, maintaining CoT monitorability has explicitly been the bigger priority for us throughout development.

   [find in text]

Keep reading

View all



Research acceleration: The view inside OpenAI

ResearchSep 6, 2026

[VIDEO OMITTED]

GPT-6 Astra: A new generation of intelligence

ResearchSep 3, 2026



Safety overview: GPT-6 Astra

SafetySep 3, 2026